Claude 3D Animation Master Prompts

Nov 6, 2026 · @Huzaifa

How to use this file

These are every prompt used to make a 3D-world, 2D-character animated short with Claude, in the order you run them. Paste them one at a time into the same project, and replace anything in [brackets].

What you need

- Claude with file and command access (the Claude desktop app with a local session, or Claude Code). Plain chat can't download videos or run code.
- Node.js and ffmpeg on your computer. Claude can install the rest.
- A laptop or PC with a dedicated GPU (NVIDIA or AMD) for fast rendering. Without one it still works, just slower.

The 4 stages

1. Train Claude on a reference style
2. Write the story and script
3. Build the visuals (the longest stage)
4. Render fast on your own GPU

Rules that make every prompt work

- Every prompt saves its output to a file. Later prompts read those files.
- Ask for numbers, not opinions. You can copy a number; you can't copy "looks warm".
- Test small first: one still frame, then a 5-second clip, then the full video.
- Judge with your own eyes on a side-by-side. Measurements can pass while the picture still looks wrong.
- Never let Claude guess the style. When it goes wrong, give it an exact list.
- Use reference videos for analysis only. Don't re-upload them or ship their frames, and design your own characters and rooms.

Stage 1: Train Claude on the style

This stage turns 10 to 20 reference videos into one rulebook file that every later prompt follows.

1.1 Set up the project

```
Set up this project folder for making an animated short.
Check that Node.js, ffmpeg, yt-dlp and Python are installed, and install
anything missing.
Create these folders: /references, /analysis, /script, /audio, /build,
/stills, /output.
Create a file called PROGRESS.md where you log every step you finish and
every problem you hit.
Tell me what you installed and what failed.
```

1.2 Download the references

```
Download these YouTube Shorts into /references, one folder per video, using
yt-dlp:
[paste 10-20 links from one channel]
For each one, download the video and the ORIGINAL language audio track (not a
dub).
Save the title, views and upload date to /references/list.csv.
These are for analysis only. Do not edit or re-upload them.
```

1.3 Measure the style (the most important prompt)

```
Analyze every video in /references. Do not give opinions. Give numbers.

1. Extract every frame at 30 fps for each video.
2. Build contact sheets: one image per 2 seconds, with all frames in that
window on a single image, so you can see small changes.
3. Look at the contact sheets yourself, not just the code output.
4. Measure per video and as an average:
   - Camera movement per frame (use phase correlation) and the percentage of
frames where the camera moves
   - Number of hard cuts and the average length of a shot
   - Brightness, contrast and saturation
   - Whether there are outlines, how thick, and what color
   - Shadow color and highlight color, and how they change across the story
   - Character proportions (head size vs body, limb thickness)
   - Background blur and depth
5. Transcribe the audio. Measure words per second and dead air.
6. Find the script pattern: how the first sentence works, how it escalates,
where the pivot is, how it ends, and any repeated phrases.

Save everything as /analysis/rulebook.md with a table of numbers and a short
"rules to follow" list.
For each number, note how you measured it and how confident you are.
At the end, list anything you could NOT measure well.
```

1.4 Measure the characters

```
Use the reference videos in the references folder.
1. Extract frames and pick 40 that show a character in a medium or close
shot, spread across all videos. Save them to stills/ref_samples/.
2. On each frame, measure:
   - Head height / total figure height, and head width / head height
   - Outline thickness as a ratio of head width, and outline color
   - Limb thickness vs head width, and the shape of hands and feet
   - Approximate camera field of view and camera height relative to the
character
   - Foreground vs background sharpness (Laplacian variance)
   - Top 6 dominant colors (hex) and average saturation
3. Report the median and the min-max range for each number. Flag outlier
frames.
4. Add a "Character spec" section to analysis/rulebook.md using the medians,
with a confidence level for each number.
```

Open 3-4 of the sample frames yourself before it measures. Title cards or wide shots with no character will throw the numbers off.

1.5 Re-check Claude's own assumptions

```
Re-check every claim in /analysis/rulebook.md.
Pick the 5 claims the build depends on most, re-measure them a different way,
and tell me which ones were wrong or only partly true.
Update the rulebook with the corrections.
```

Stage 2: Story and script

Pick a true story the reference channel hasn't told, then write it in their exact script formula.

2.1 Find a story and fact-check it

```
Read /analysis/rulebook.md and the transcripts.
1. List every topic the reference channel already covered.
2. Suggest 8 new stories that fit the same formula (a strange fact that
sounds impossible in the first sentence, then the real reason). They must be
true and well documented.
3. For your top 3, check every fact (dates, names, numbers, places) and give
me the sources.
4. Tell me which one has the strongest first line and why.
Save to /script/story_options.md. Do not write the script yet.
```

2.2 Write the script

```
Use story option [number] and the script pattern from the rulebook.
Write a [30-45] second voiceover script.
Rules:
- Start with a paradox in the first sentence. No intro.
- Follow the same escalation, pivot and payoff as the references.
- Match the words-per-second in the rulebook.
- Simple words, human tone, no generic AI phrases.
- Use only facts we verified.
Split it into numbered shots, each with the line spoken and a one-line
description of what's on screen.
Save to /script/script_v1.md.
```

Edit the script in your own voice, save it as /script/script_final.md, then generate the voiceover (ElevenLabs or your usual tool) as /audio/voiceover.wav.

2.3 Time the voiceover

```
Transcribe /audio/voiceover.wav with word-level timestamps.
Match each line of /script/script_final.md to its start and end time.
Save /audio/timing.json with: shot number, line, start time, end time,
duration.
Flag any shot shorter than 1.5 seconds.
```

Lock the script before you record. If you change it later, the timing changes and you have to redo this step.

Stage 3: Build the visuals

Get one still frame right first, then a 5-second clip, then the full video. Skipping that order is where most of the time gets wasted.

3.1 The 5-second test

```
Using only /analysis/rulebook.md, build a 5-second test scene with Three.js
and Remotion. Do not use Blender.
- One flat 2D character in a real 3D environment
- Ink outlines on the 3D edges
- One slow camera push with easing, using the camera numbers from the
rulebook
- Lighting and colors from the rulebook
Before rendering the video, render 5 still frames to /stills and check them
against the rulebook.
Fix problems first (camera inside objects, wrong colors, anything blocking
faces), then render the clip to /build/test.mp4.
Log what went wrong in PROGRESS.md.
```

Your first result will probably look cheap. That's normal.

3.2 Still-frame check (run before every render)

```
Render one still frame per shot from the current build to /stills.
Check each against the rulebook and list problems: camera inside geometry,
objects blocking faces, colors off by more than 10%, missing outlines, wrong
lighting.
Fix everything you find, re-render the stills, and show me before and after.
Do not render any video until the stills pass.
```

3.3 Match one frame first (the trick that works)

```
Do not make a video yet. Recreate one reference shot as a calibration test:
[describe the shot, e.g. the kitchen scene at 0:12 in video 3]. Use our own
character, not theirs.
Build in this order, and render a still after each step so I can judge it:
1. Room: walls, trim, windows, furniture, floor in correct perspective, small
props. Paint every texture in code.
2. Light: one warm key light, soft cool fill, soft shadows under the
character and props, a gentle vignette, background blur.
3. Character: hair, clothing detail, a body shape like the reference, and a
pose that touches a prop.
4. Camera: low, close, slightly off-center, with a foreground object
overlapping the frame.
After each step, save a side-by-side with the reference to stills/ and list
what is still different, in plain words, before fixing it.
```

3.4 Style fix rounds (repeat until it matches)

Use whichever lines match what you see in the side-by-side.

```
Compare against the reference frame side by side after each step. Judge by
eye, not by measurements.
- Flat style: remove grain, gloss, bevels and reflections. Use flat color
fills with soft cel shading and a gentle gradient on walls.
- World ink: add dark ink outlines on every prop edge, clearly visible at
phone size, thinner than the character outline.
- Character outline: measure the outline on the reference separately for
head, body and limbs, and match it. Never go thicker than the reference.
- Character detail: hair as a distinct cap with a few strands and a
highlight, eyebrows with an expression, glasses or accessories with frame
detail, clothing detail.
- Grounding: both feet on the floor with a small soft contact shadow. No
floating.
- Camera: closer and lower, character fills about 50% of the frame height.
- Light: a soft, semi-transparent warm light patch from the window, so the
floor pattern still shows through.
- Props: rebuild any boxy prop as a real model with its parts (for a phone:
body, dial, handset, coiled cord, bracket).
Save a side-by-side and a zoomed crop of the character. List what still
differs. No video yet.
```

If the shading looks too strong, add: *"Keep the shading subtle: about 10-12% lightness shift between highlight and shadow, same hue, no shift toward orange or red. Highlight is a small soft patch on the light side only."*

3.5 Motion test

```
The still is approved. Render a 5-second motion test of this scene to
build/motion_test.mp4 at 1080x1920, 30 fps.
- Camera: slow push-in over the full 5 seconds with ease in and ease out. No
hard cuts.
- Character: two natural blinks, a small breathing bob, a slight head tilt,
and one small hand gesture. Small secondary motion on hair and cloth.
- World: a very slow drift of the light.
- Keep all outlines, shading and colors exactly as in the approved still.
Render 6 stills from across the clip and check for outline flicker, popping
and camera clipping first.
Then compare with a 5-second reference clip and list the differences in
camera speed, easing and character movement.
```

3.6 The no-guesswork style spec (use this if Claude falls back to flat 2D)

If you only say "match the style", Claude may take the easy way and make a flat side-view animation. This prompt removes the guessing.

```
Study the reference videos again, as video and not as still frames:
1. Backgrounds: every background is a real 3D world made of detailed 3D models (buildings, props, furniture, ground) with painted flat-color textures and dark ink outlines. The 2D characters stand inside this world and are lit by it. No flat sky-and-ground strips, no cardboard side-on buildings.
2. Camera: the camera travels through the 3D world: push-ins, tracking, orbits, crane moves, moving from one subject to the next without a cut. Measure average camera speed and how many shots have travel vs a static camera, and add it to the rulebook.
3. Depth: every shot has a foreground, midground and background. Foreground objects pass close to the camera; the background is slightly blurred; the character stays sharp.

Rebuild the video in that style. Keep exactly the same story, script, voiceover, timing, characters and on-screen facts.
- Build each location as a real 3D scene in Three.js with detailed models.
- Texture all models with painted flat colors and wear, and outline every object edge in ink.
- Every shot has camera movement through the 3D space. Plan each move to follow the story.
- Put on-screen labels inside the world (a signpost, a banner, a painted sign), not floating.
- Keep characters large enough to act in medium and close shots.

Stop after each step for my approval:
1. Write the camera plan: for every shot, the location, camera path (start, end, speed) and key objects.
2. Render one still per shot at the start and end of its camera path, as a contact sheet.
3. After I approve, render a 5-second test of the hardest shot with the full camera move.
4. After I approve the test, render the full video with the voiceover.
```

3.7 Full build

```
Build the full short from /script/script_final.md and /audio/timing.json.
- One scene per shot, timed exactly to the voiceover
- Same character, outline, shading, room style and motion rules as the approved test
- Our own rooms and props for each location, not copies of the reference
- Mostly slow camera moves, at most one hard cut in the whole video
- Lighting that follows the story: warm and bright during the setup, deeper blue at the pivot
- 3 to 5 face expressions that match the script's emotion
Render stills for every shot first and show me a contact sheet. Do not render video until I approve the stills.
Then render to output/v1.mp4 with the voiceover attached.
```

3.8 Find and fix mistakes

```
The rendered video has mistakes. Don't patch randomly. Review first, then fix shot by shot.
1. Review our render frame by frame with a contact sheet for every 2 seconds. Check every shot for:
   - Style: real 3D world with detail, consistent ink outlines
   - Camera: smooth travel with easing; no static shots, jumps, shaking or clipping inside objects
   - Characters: correct size and design, grounded with a contact shadow, no sliding or floating, no limbs without a body
   - Depth: foreground, midground, background; background blurred, character sharp
   - Objects: nothing floating, intersecting, flickering, popping, stretched or missing; no black voids under buildings; every prop rests on something or is mounted
   - Text and facts: every date, name and number matches the script and is spelled right
   - Timing: shots match the voiceover; no blank or dead frames
2. Write a numbered list of every mistake with timestamp, shot, problem and severity. Save it to analysis/mistakes_round_1.md and show me.
3. Fix high severity first, then medium, one shot at a time, with a before/after still for each.
4. Re-render only the shots that changed, join them with the voiceover, and review again.
Don't tell me it's fixed until you have checked the new frames.
My notes: [your timestamps and problems]
```

3.9 Make the animation feel real

```
Keep the story, timing, characters and style. Make the animation more
realistic:
- The main event: show it clearly, step by step, with physical cause and
effect (small steps, a slight shudder, dust and debris).
- Characters: walk cycles with real foot placement, weight shifts, breathing,
blinks, head turns, and gestures that match the voiceover. Characters react
to what happens.
- Secondary motion: curtains, cloth, hair, ropes, water ripples, dust in the
light.
- Camera: smooth moves with ease in and out, a very small natural sway, no
sudden tilts unless the story needs one.
- Lighting: soft shadows, light shafts through windows, a slow change in time
of day across the video.
- Background people who react, so the world feels alive.
Do the main event first, render a 3-second clip for my approval, then do the
rest.
```

Stage 4: Render fast on your own GPU

Claude's cloud machine has no graphics card, so 3D renders there are slow. Let Claude write the code, and let your own PC do the rendering.

Setup checklist

1. Install Node.js and ffmpeg.
2. Open the Claude desktop app and start a **local** session in a folder on your computer, not a cloud session. Only a local session can use your hardware.
3. Put your project folder there.
4. In Windows Settings > System > Display > Graphics, set Chrome and Node to **High performance** (your NVIDIA or AMD card). Laptops use the weak built-in chip by default, and this step catches most people.
5. While rendering, open Task Manager > Performance > GPU to confirm the GPU is busy.

4.1 Check the session is really on your PC

```
Run commands that show my computer's operating system, GPU name, GPU driver
version, CPU, and RAM. Tell me what you found, and say whether this looks
like my real computer or a virtual machine. Do not guess.
```

If it says "virtual machine" or shows no GPU, you're in a cloud session. Start a new local one.

4.2 Set up GPU rendering

```
This project renders with Remotion and Three.js. Set it up so rendering uses
my GPU:
1. Run npm install to download Remotion and the dependencies.
2. Configure the Remotion render to use GPU-accelerated WebGL (try the ANGLE
setting first, and tell me which setting worked).
3. Render a 5-second test shot with GPU on and with GPU off, and tell me both
render times.
4. Make a render.bat file I can double-click to render the full video, plus
one command per shot.
5. If the GPU is not being used, tell me why and what to fix (drivers,
browser, flags).
```

4.3 Make rendering faster

```
Make rendering faster without changing how the video looks:
1. Add a preview mode: 540x960, JPEG frames, low shadow quality, no post-
processing. Use it for every motion test.
2. Render each shot to its own file in build/shots/, so I can re-render one
shot without touching the others, and join them with ffmpeg at the end.
3. Profile one 5-second shot and tell me which part takes the most time (ink
outline, blur, shadows, textures, geometry). Fix the top two.
4. Instance repeated objects and reduce texture sizes to 2048 or lower.
5. Add a final-render option: render at 720x1280, upscale to 1080x1920 with
ffmpeg, and compare a still to a native 1080 render so I can check there's no
visible quality loss.
Report render time before and after for the same 5-second shot.
```

Extras

Continue in a new session

Paste this when you move the project to a new chat or a local session, so Claude picks up where you left off.

```
This is a continuing project. Read all files in this folder first, especially
analysis/rulebook.md and PROGRESS.md.
Project: a 9:16 animated short, about [length] seconds, story: [one line].
Keep the existing script, voiceover and timing exactly.
Approved style: a real 3D world built from detailed models in Three.js,
painted flat-color textures, dark ink outlines on every edge. 2D characters
inside the world with the approved proportions, outline and subtle shading. A
camera that travels through the world with ease in and out. Foreground,
midground and background in every shot; background slightly blurred,
character sharp.
How we work: camera plan first, then one still per shot for my approval, then
a 5-second test, then the full render. Judge by eye against the reference,
not just measurements.
First task: [what you want next].
```

Package your work as a reusable skill

```
Turn /analysis/rulebook.md, our prompts, and the fixes we learned into one
Claude skill.
Include: what the skill does, step-by-step instructions, the rulebook, the
corrections, and a short "common mistakes" section.
Write a README that says plainly what the skill needs (Claude access,
Node.js, ideally a GPU) and what it cannot do.
Do not include downloaded videos or frames from other creators.
```

Troubleshooting

Problem you see	What to tell Claude
Everything is made of boxes	Run 3.3: match one reference frame side by side before any video
Room looks like a realistic 3D render	Remove grain, gloss and reflections; flat fills with soft cel shading (3.4)
Character looks pasted on	Add a soft contact shadow, match the outline per body part, add hair and clothing detail (3.4)
Outline too thin or too thick	Measure the reference outline for head, body and limbs separately and copy those values
Video came out flat 2D	Run 3.6, the no-guesswork style spec

Problem you see	What to tell Claude
Objects float or black gaps show	Run 3.8 and list every floating object and void with its timestamp
Animation feels stiff	Run 3.9, starting with the main event
Render takes forever	Do Stage 4: local session, GPU on, preview mode, one file per shot
Numbers pass but it still looks wrong	Trust your eyes. Send a side-by-side and describe the difference in plain words